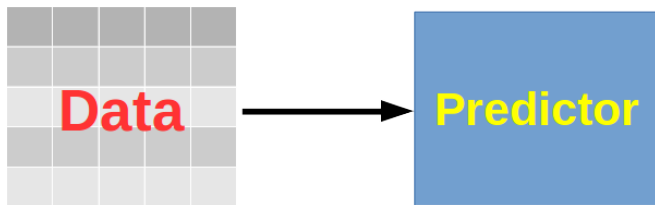


Feature Engineering

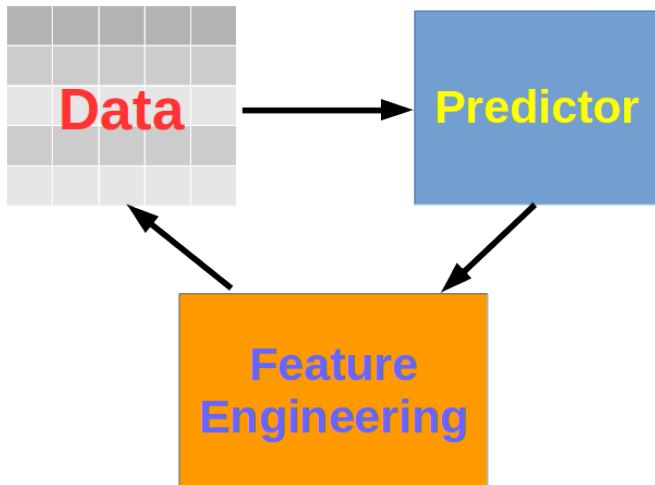
Zhirong Yang

At the beginning

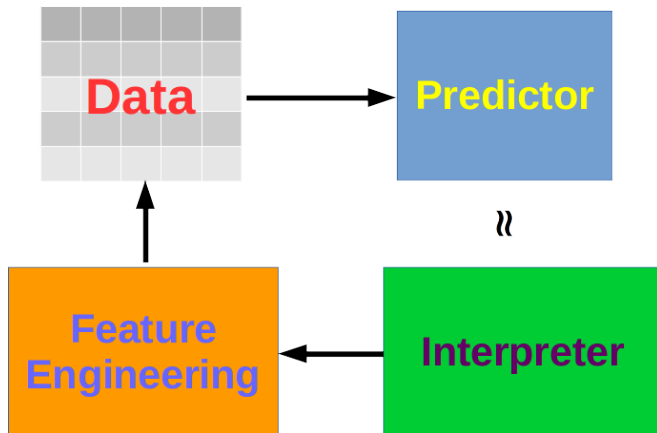
The data is in raw features



Feature engineering helps prediction



Feature engineering also helps interpretation



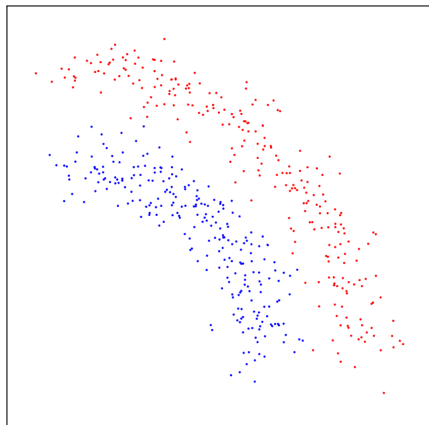
Feature Engineering

- ▶ Feature selection
 - ▶ Delete some existing columns
 - ▶ Can be done by using feature importance
- ▶ Feature generation (feature extraction)
 - ▶ Add some new columns
 - ▶ Generate new attributes using existing ones for each row
 - ▶ Generate new attributes across multiple rows

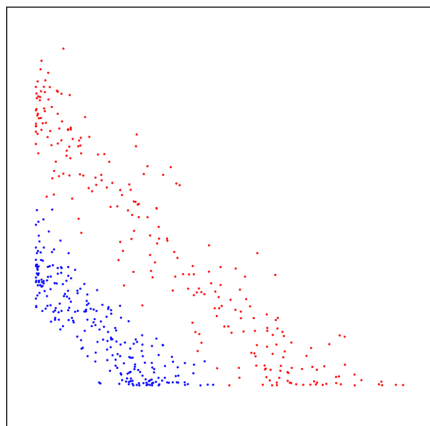
Feature Engineering

- ▶ Feature selection
 - ▶ Delete some existing columns
 - ▶ Can be done by using feature importance
- ▶ Feature generation (feature extraction)
 - ▶ Add some new columns
 - ▶ Generate new attributes using existing ones for each row
 - ▶ Generate new attributes across multiple rows
- ▶ Automatic feature engineering is an open problem!
 - ▶ The top conference ICLR mainly addresses this

Why new features can help? Example 1

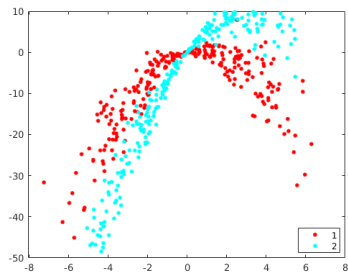


x_1 vs x_2

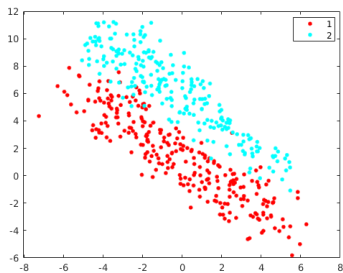


x_1^2 vs x_2^2

Why new features can help? Example 2



X_1 VS X_2



X_1 VS $\frac{X_2}{X_1}$

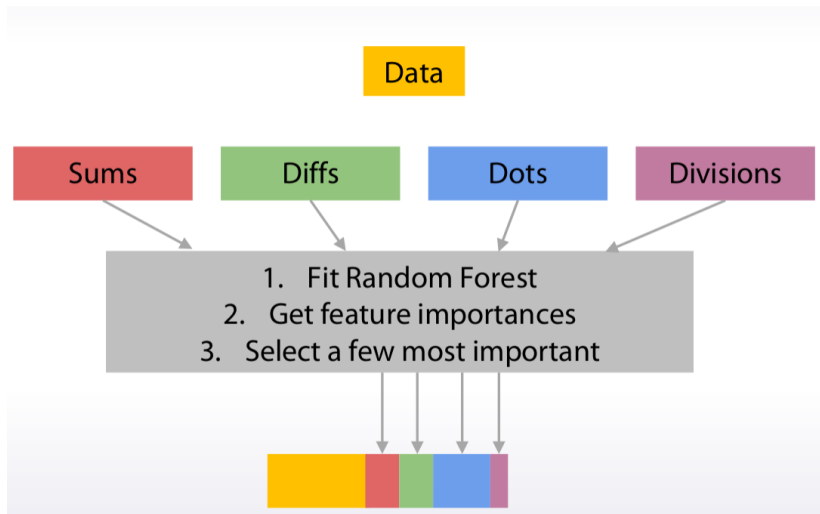
Row-wise feature generation

- ▶ Individual transform, e.g.
 - ▶ x^2
 - ▶ $\log(1 + x)$
- ▶ Pairwise transform, e.g.
 - ▶ $x + y$
 - ▶ $x - y$
 - ▶ $x \cdot y$
 - ▶ x / y
- ▶ Sometimes involve larger groups
 - ▶ mean, std, min, max, percentile, etc.

Practical notes

- ▶ We have a lot of possible interactions — d^2 for d features.
- ▶ Even more if use several types in interactions
- ▶ Need to reduce number of features
 - ▶ Group existing features before generation
 - ▶ Feature selection after generation

Example of interaction generation pipeline



High-order linear interactions

- ▶ Principal Component Analysis (PCA)
 - ▶ Finds uncorrelated components
- ▶ Canonical Correlation Analysis (CCA)
 - ▶ Finds correlated components across multiple sources

Principal Component Analysis (PCA)

- ▶ We have seen PCA in the Unsupervised Learning lecture
- ▶ Do PCA in Python

Wrong way

```
1 pca = PCA(n_components=5)
2 X_train_pca = pca.fit_transform(X_train)
3 X_test_pca = pca.fit_transform(X_test)
```

Right way

```
1 X_all = np.concatenate([X_train, X_test])
2 pca.fit(X_all)
3 X_train_pca = pca.transform(X_train)
4 X_test_pca = pca.transform(X_test)
```

Canonical Correlation Analysis (CCA)

- ▶ Multiple aspects of the same data object
 - ▶ e.g. article (text, author), webpage (text, hyperlinks), TV news (vision, speech, scripts), song (sound, lyrics)

Canonical Correlation Analysis (CCA)

- ▶ Multiple aspects of the same data object
 - ▶ e.g. article (text, author), webpage (text, hyperlinks), TV news (vision, speech, scripts), song (sound, lyrics)
- ▶ CCA seeks linear transforms such that correlation is maximized in the common subspace
 - ▶ X and Y are random (vector) variables

$$(a', b') = \operatorname{argmax}_{a, b} \operatorname{corr}(a^T X, b^T Y)$$

Canonical Correlation Analysis (CCA)

- ▶ PCA is implemented by eigendecomposition

$$\Sigma w = \lambda w$$

- ▶ CCA is implemented by generalized eigendecomposition

$$\begin{pmatrix} 0 & \Sigma_{XY} \\ \Sigma_{YX} & 0 \end{pmatrix} \begin{pmatrix} w_X \\ w_Y \end{pmatrix} = \lambda \begin{pmatrix} \Sigma_{XX} & 0 \\ 0 & \Sigma_{YY} \end{pmatrix} \begin{pmatrix} w_X \\ w_Y \end{pmatrix}$$

Canonical Correlation Analysis (CCA)

- ▶ PCA is implemented by eigendecomposition

$$\Sigma w = \lambda w$$

- ▶ CCA is implemented by generalized eigendecomposition

$$\begin{pmatrix} 0 & \Sigma_{XY} \\ \Sigma_{YX} & 0 \end{pmatrix} \begin{pmatrix} w_X \\ w_Y \end{pmatrix} = \lambda \begin{pmatrix} \Sigma_{XX} & 0 \\ 0 & \Sigma_{YY} \end{pmatrix} \begin{pmatrix} w_X \\ w_Y \end{pmatrix}$$

- ▶ In Scikit-Learn, example:

```
1 from sklearn.cross_decomposition import CCA
2 X = [[0., 0., 1.], [1., 0., 0.], [2., 2., 2.], [3., 5., 4.]]
3 Y = [[0.1, -0.2], [0.9, 1.1], [6.2, 5.9], [11.9, 12.3]]
4 cca = CCA(n_components=1)
5 cca.fit(X, Y)
6 X_c, Y_c = cca.transform(X, Y)
```

Cross-row feature generation: example

Original data

	User_id	Page_id	Ad_price	Ad_position
0	4	6	165.977125	Bottom_right
1	4	6	34.5395640	Bottom_right
2	4	6	29.1963786	Bottom_left
3	4	6	79.4373729	Bottom_left
4	4	6	290.534595	Bottom_right
5	4	6	314.412660	Bottom_right
6	4	6	138.9007639	Bottom_right
7	4	6	107.4711914	Bottom_right
8	4	6	242.1089786	Bottom_left
9	4	7	27.16719836	Bottom_left
10	4	7	413.5421978	Bottom_right

Cross-row feature generation: example

Augmented data

	User_id	Page_id	Ad_price	Ad_position	Max_price	min_price	Min_price_position
0	4	6	95.874252	Bottom_right	474.63772	73.711548	Bottom_left
1	4	6	215.751007	Bottom_right	474.63772	73.711548	Bottom_left
2	4	6	474.637726	Bottom_left	474.63772	73.711548	Bottom_left
3	4	6	73.711548	Bottom_left	474.63772	73.711548	Bottom_left
4	4	6	79.288841	Bottom_right	474.63772	73.711548	Bottom_left
5	4	6	271.391785	Bottom_right	474.63772	73.711548	Bottom_left
6	4	6	296.529053	Bottom_right	474.63772	73.711548	Bottom_left
7	4	6	96.030029	Bottom_right	474.63772	73.711548	Bottom_left
8	4	6	130.175064	Bottom_left	474.63772	73.711548	Bottom_left
9	4	7	35.465202	Bottom_left	121.54219	35.465202	Bottom_left
10	4	7	121.542191	Bottom_right	121.54219	35.465202	Bottom_left

Cross-row feature generation: example

```
In [22]: gb = df.groupby(['user_id', 'page_id'], as_index=False).agg(
          {'ad_price': {'max_price': np.max, 'min_price': np.min}})
          gb.columns = ['user_id', 'page_id', 'min_price', 'max_price']
          gb
```

Out[22]:

	user_id	page_id	min_price	max_price
0	4	6	73.711548	474.637726
1	4	7	35.465202	121.542191

```
In [23]: df = pd.merge(df, gb, how='left', on=['user_id', 'page_id'])
```

More features

- ▶ How many pages user visited
- ▶ Standard deviation of prices
- ▶ Most visited page
- ▶ etc.

Local features

- ▶ Many datasets contain time or spatial information
- ▶ Various temporal and/or spatial features
 - ▶ Define windows (neighborhood) at various scales
 - ▶ Calculate statistics within the windows (neighborhoods)

Example: KNN features for Springleaf data set

- ▶ For every point, find 2000 nearest neighbors using Bray-Curtis metric

$$d(u, v) = \sum |u_i - v_i| / \sum |u_i + v_i|$$

- ▶ Calculate various features from those 2000 neighbors
 - ▶ Mean target of nearest 5, 10, 15, 500, 2000 neighbors
 - ▶ Mean distance to 10 closest neighbors
 - ▶ Mean distance to 10 closest neighbors with target 1
 - ▶ Mean distance to 10 closest neighbors with target 0